

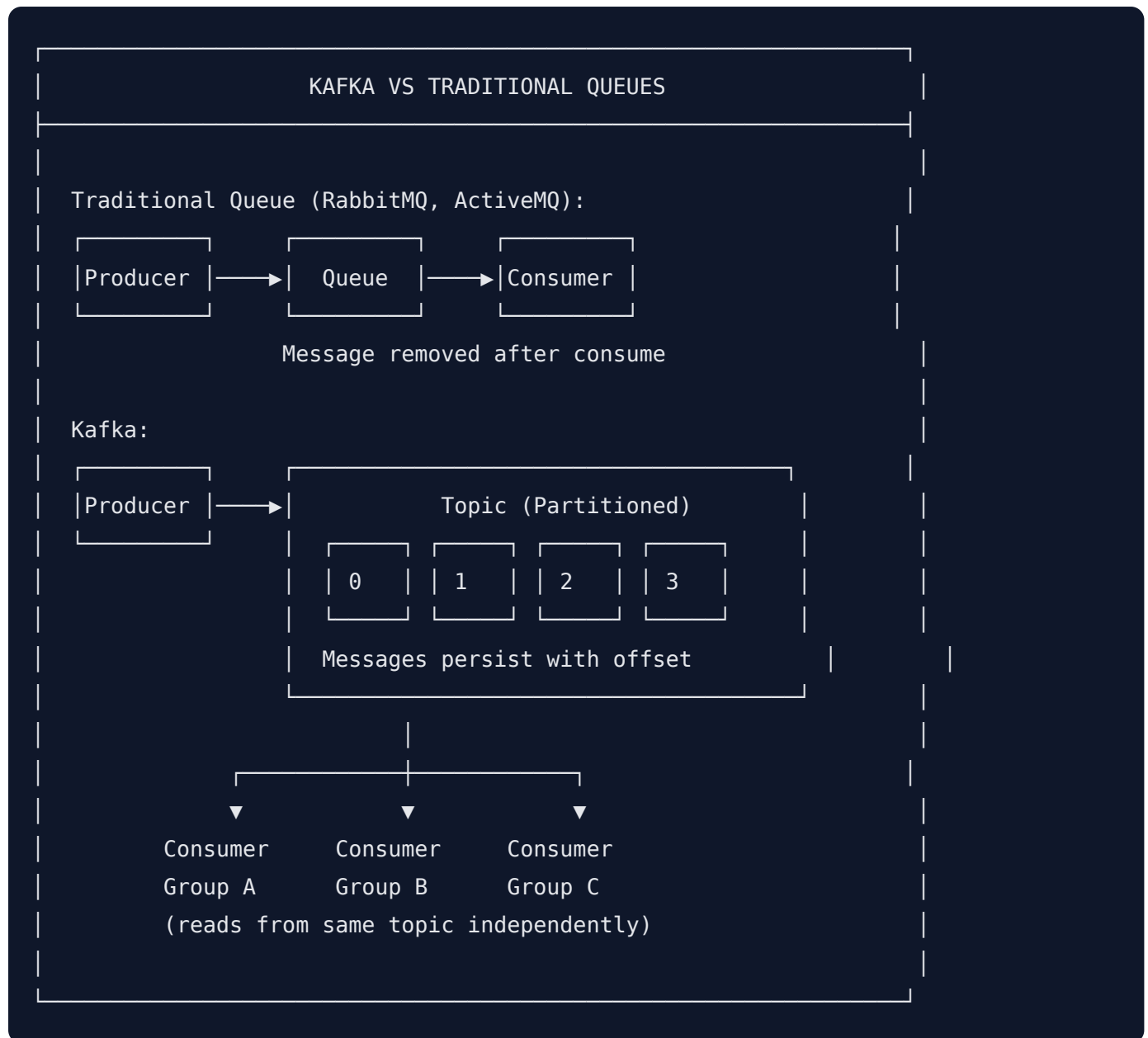
Here is your **Kafka Deep Dive** — comprehensive, interview-focused, and formatted in clean markdown for PDF export.

KAFKA DEEP DIVE — Advanced Microservices Interview Guide

For the 16-Year Experience Professional

PART 1: KAFKA CORE ARCHITECTURE

What Makes Kafka Different



Kafka Terminology — Must Know

Term	Definition	Interview Importance
Topic	Logical stream of messages	★★★★
Partition	Ordered, immutable sequence of messages	★★★★
Offset	Unique ID per message within partition	★★★★
Broker	Kafka server (one node in cluster)	★★★
Producer	Publishes messages to topics	★★★★
Consumer	Reads messages from topics	★★★★
Consumer Group	Multiple consumers reading together	★★★★

Leader/Follower	Partition replication for HA	★★
ISR	In-Sync Replicas	★★★
Log Segment	Physical file storing messages	★

PART 2: PARTITIONING DEEP DIVE

How Partitioning Works



Partitioning Strategies Comparison

Strategy	How It Works	Use Case	Ordering Guarantee
Default (Round Robin)	Cycle through partitions	High throughput, no key requirement	No ordering across partitions

Keyed	hash(key) % num_partitions	Need ordering per entity (e.g., per customer)	Per-key ordering preserved
Custom Partitioner	Custom logic	Complex routing, sticky partitioning	Depends on implementation
Sticky Partitioner	Batch to same partition, switch when full	Optimal batching + reduced latency	No ordering guarantee

Partition Count Decisions

PARTITION COUNT FORMULA (Rough Rule)

$\text{Max Partitions} = \text{Target Throughput} / \text{Max Throughput per Partition}$

Example:

- Target: 100 MB/s total throughput
- One partition: ~10 MB/s max
- Partitions needed = $100 / 10 = 10$

WARNINGS:

- More partitions → more file handles
- More partitions → slower leader election
- More partitions → higher latency for small messages
- More partitions → more consumer rebalancing time

Rule of thumb: 1 partition = 10 MB/s throughput

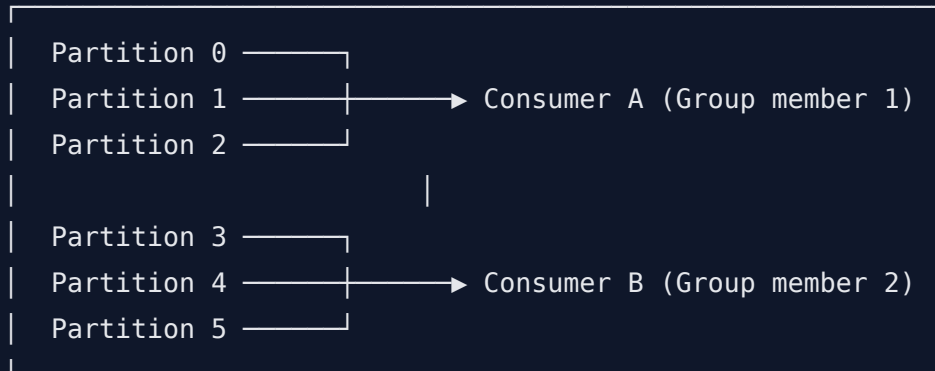
Sweet spot: 100-200 partitions per broker max

PART 3: CONSUMER GROUPS & REBALANCING

Consumer Group Architecture

CONSUMER GROUP – DETAILED

Topic: "orders" with 6 partitions



RULES:

1. Each partition assigned to exactly ONE consumer in group
2. One consumer can handle multiple partitions
3. Max consumers in group = number of partitions
(Extra consumers sit idle)

Rebalancing Trigger:

- Consumer joins or leaves group
- Consumer commits too slowly (session timeout)
- Partition count changes
- Topic reassignment happens

Rebalancing Protocols

Protocol	How It Works	Pros	Cons
Eager Rebalancing (Classic)	All consumers revoke all partitions → Redistribute	Simple, predictable	Stop-the-world during rebalance
Cooperative (Incremental)	Only revoke some partitions, reassign gradually	Minimal pause, faster	Complex, newer

Sticky Assignor	Keep previous assignments when possible	Less movement	Implementation complexity
-----------------	---	---------------	---------------------------

Consumer Heartbeat & Session Timeouts

HEARTBEAT CONFIGURATION DEEP DIVE

Config:

```
session.timeout.ms = 45000 (45 sec default)
heartbeat.interval.ms = 3000 (1/3 of session)
max.poll.interval.ms = 300000 (5 min default)
```

TIMELINE OF FAILURE:

```
t=0s: Consumer receives message, starts processing
t=45s: No heartbeat sent (processing still ongoing)
t=46s: Broker marks consumer dead → Rebalance
t=47s: Original consumer finishes processing
      → Now processing same message again!
```

Interview Insight:

```
"Long processing needs increased max.poll.interval.ms,
not increased session.timeout.ms"
```

PART 4: MESSAGE DELIVERY SEMANTICS

The Three Delivery Guarantees

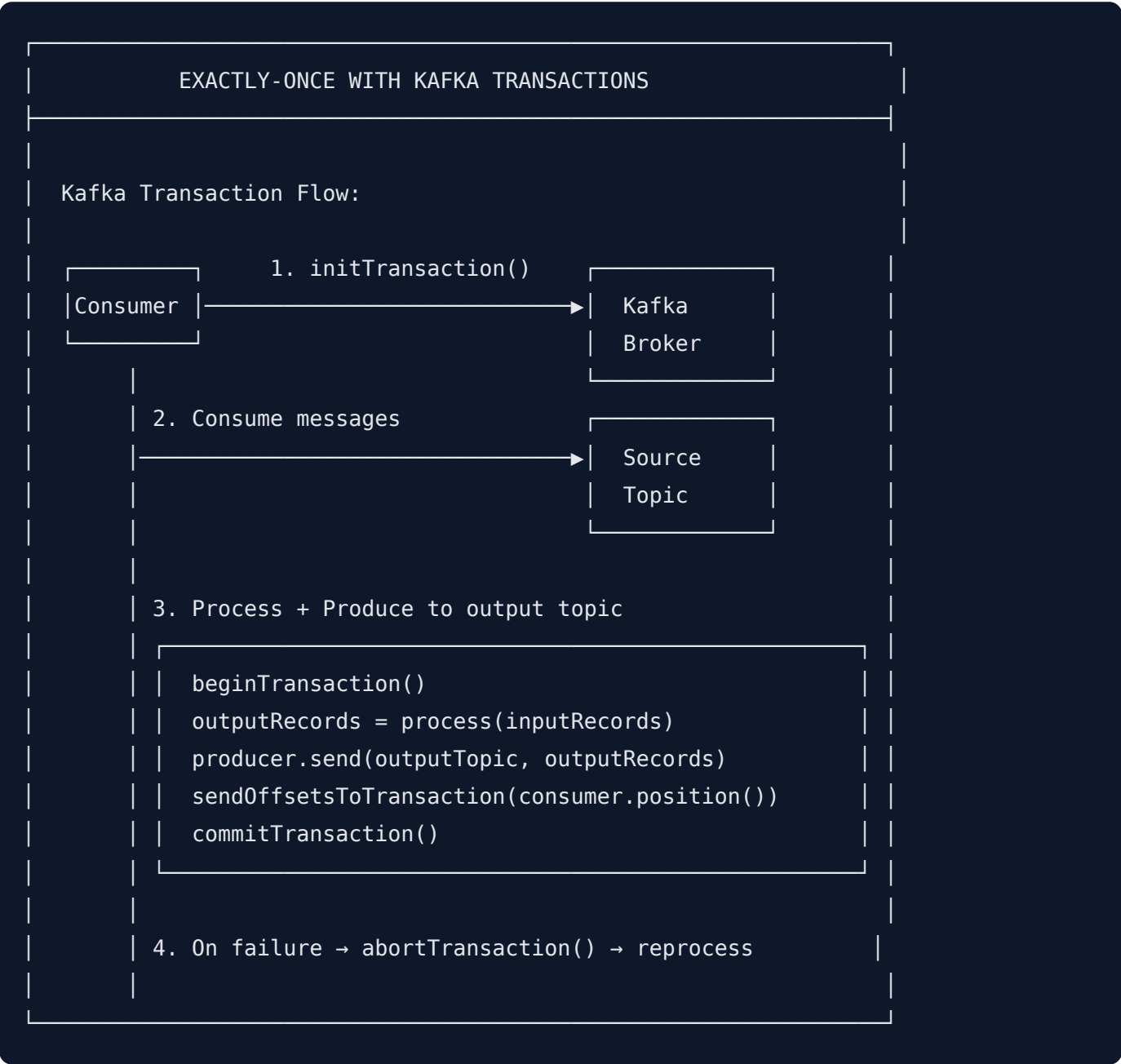


Producer Delivery Configurations

Config	Value	What It Does	Delivery Guarantee
acks=0	Producer doesn't wait	Fastest, no guarantee	At-most-once
acks=1	Leader only	Low latency, risk on leader fail	At-least/At-most

<code>acks=all</code>	All ISRs	Strongest durability	At-least-once
<code>enable.idempotence=true</code>	Exactly-once within partition	Prevents duplicate sends	Exactly-once per partition
<code>max.in.flight.requests.per.connection=1</code>	Single request at a time	Required for exactly-once	Ordering guarantee

Exactly-Once Processing — The Real Way



Exactly-Once Trade-offs

Aspect	At-least-once + Idempotent	Exactly-Once with Transactions
--------	----------------------------	--------------------------------

Throughput	High	30-50% lower
Latency	Low	Higher (coordinator overhead)
Complexity	Low	High
Recovery	Manual dedup needed	Automatic
When to use	Most cases	Financial, critical counting

Interview Insight: *"True exactly-once is expensive. Most production systems use at-least-once with idempotent consumers — same effect, better performance."*

PART 5: IDEMPOTENT PRODUCER DEEP DIVE

How Idempotent Producer Works

IDEMPOTENT PRODUCER INTERNALS

Producer sends message with:

```
Producer ID (PID) = Unique per producer instance
Sequence Number = Increments per message per partition
Producer Epoch = Increments when PID reused
```

Broker maintains:

```
PID → Last Sequence Number (per partition)

When new message arrives:
if (sequence == last_sequence + 1) → Accept
if (sequence <= last_sequence) → Reject (duplicate)
if (sequence > last_sequence + 1) → Reject (gap)
```

Example:

```
Send(seq=1) → Accepted (last_seq=0, new_seq=1) ✓
Send(seq=2) → Accepted (last_seq=1, new_seq=2) ✓
Send(seq=2) → Rejected (duplicate) ✗
Send(seq=4) → Rejected (gap, seq=3 missing) ✗
```

PART 6: COMPRESSION & SERIALIZATION

Compression Comparison

Compression	Speed	Ratio	CPU Usage	Network Saving	Best For
None	Fastest	0%	Minimal	0%	Low throughput

gzip	Slow	Best (70-80%)	High	High	Text, batches, archives
snappy	Fast	Good (30-40%)	Low	Medium	Balanced, Google style
lz4	Fastest	Good (30-40%)	Very Low	Medium	Java ecosystem
zstd	Medium	Better than gzip (80%+)	Medium	Highest	Newer, best combo

Serialization Formats Deep Dive

SERIALIZATION FORMAT COMPARISON

JSON:

Pros: Human readable, ubiquitous

Cons: Large (20-50+ bytes overhead per message)
No schema evolution

Avro:

Schema Registry Required!

Message = schema_id (4 bytes) + binary data

Schema stored separately, only ID in message

Evolution: Add optional field → old readers ignore
Remove field → old readers use default

Best for: Long-term storage, streaming, evolved schemas

Protobuf:

Pros: Smaller than Avro? Actually similar
Schema in code (no registry required)
Multiple language support

Cons: Schema changes require code changes

Sizing Comparison (100,000 records):

JSON raw: 50 MB

JSON + gzip: 15 MB

Avro: 12 MB

Avro + snappy: 8 MB

Protobuf + lz4: 7 MB (plus codegen)

PART 7: KAFKA CONNECT & KAFKA STREAMS

Kafka Connect — Plug and Play Integration



Kafka Streams vs ksqlDB vs Flink

STREAM PROCESSING COMPARISON

Kafka Streams (Java/Scala Library)

- No additional cluster
- Exactly-once semantics
- State stores (RocksDB in consumer)
- Use when: You're in JVM, want tight integration

ksqlDB (SQL on Kafka)

- Declarative SQL
- Interactive queries (pull queries)
- Materialized views
- Use when: Analysts, simpler pipelines, SQL preferred

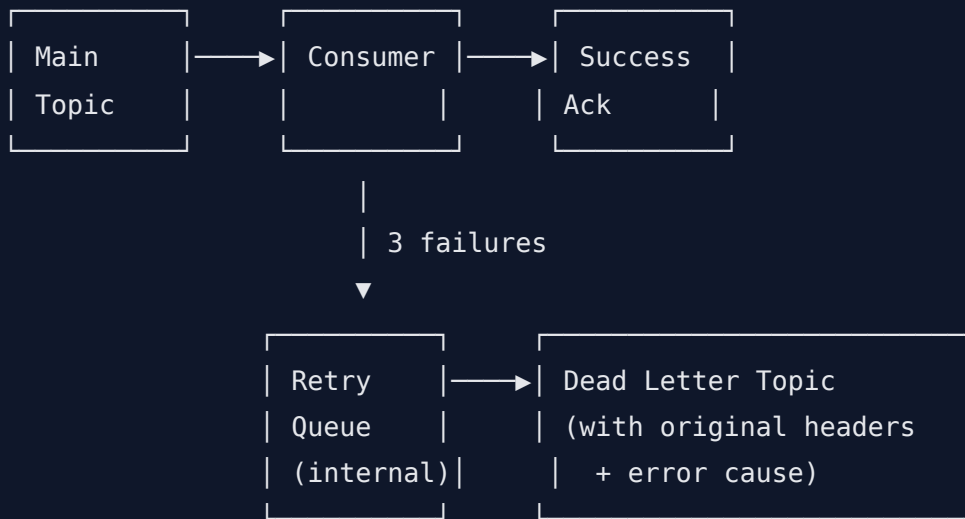
Flink (Separate processing cluster)

- True streaming (not micro-batch)
- Event time processing
- Complex event processing (CEP)
- Use when: Low-latency (sub-second), Python, complex

PART 8: DEAD LETTER QUEUES (DLQ) & ERROR HANDLING

DLQ Architecture

DEAD LETTER QUEUE PATTERN



Retry Strategy:

Attempt 1: Immediate
Attempt 2: Delay 100ms
Attempt 3: Delay 500ms
Attempt 4: Delay 2s
Attempt 5: Delay 10s → DLQ after this

DLQ Consumer (Separate team/app):

- Alerts on DLQ messages
- Manual inspection
- Repair and republish
- Dead letter → reprocessed or archived

Spring Kafka DLQ Configuration

```
// Interview-ready code understanding
@RetryableTopic(
    attempts = "4",
    backoff = @Backoff(delay = 1000, multiplier = 2),
    topicSuffixingStrategy = TopicSuffixingStrategy.SUFFIX_WITH_INDEX_VALUE,
    dltTopicSuffix = ".dlq"
)
@KafkaListener(topics = "orders")
public void consume(Order order) {
    // processing logic
}
```

PART 9: MONITORING & PRODUCTION TUNING

Critical Kafka Metrics

MUST-MONITOR KAFKA METRICS

Broker Metrics:

- Under-replicated partitions (should be 0)
- Offline partitions (should be 0)
- ISR shrink/expand rate (stable)
- Request handler avg idle (>30% is healthy)
- Network handler avg idle
- Log flush rate

Producer Metrics:

- request-latency-avg (target: <100ms)
- record-queue-time-avg (check batching)
- record-error-rate (spikes = issues)
- compression-rate (target: >0.3)

Consumer Metrics:

- records-lag-max (most important!)
- records-lag-avg
- fetch-latency-avg
- commit-latency-avg
- rebalance rate

Lag Monitoring Alert Thresholds

Metric	Warning	Critical	Action
Consumer lag	>10,000	>100,000	Scale consumers, increase partitions
Under-replicated partitions	>0 for 5 min	>0 for 30 min	Network or broker issue
ISR count	<2 for partition	=1 for partition	Broker failure, urgent

Producer error rate	>1%	>5%	Check serialization, timeout config
Rebalance frequency	>1/hour	>5/hour	Check session timeout config

PART 10: KAFKA TROUBLESHOOTING — REAL SCENARIOS

Scenario 1: Consumer Lag Growing Forever

Symptoms:

- Lag increasing monotonically
- Consumer processing not keeping up

Root Causes:

1. Slow processing (DB bottleneck, external API)
2. Single consumer with max.poll.records too high
3. Not enough partitions (parallelism limit)
4. Head-of-line blocking with large messages

Solutions:

- Solution A: More consumers (if partitions available)
- Solution B: Increase max.poll.records (batch size)
- Solution C: Increase partitions (with careful planning)
- Solution D: Async processing with manual offset commit
- Solution E: Co-partition join optimization

Scenario 2: Rebalancing Too Often

Symptoms:

- "Rebalancing" logs every few minutes
- Processing pauses frequently

Root Causes:

1. Consumer processing > max.poll.interval.ms
2. Network issues causing heartbeat loss
3. Garbage collection pauses > session.timeout.ms
4. Consumer group metadata updates too frequent

Diagnostic:

Check consumer logs for:

"Member xxx has left group" → session timeout

"Revoking partitions" → triggered rebalance

Solutions:

- Increase max.poll.interval.ms (if long processing)
- Increase session.timeout.ms (if GC/network issues)
- Use cooperative rebalancing (rebalance protocol)
- Move to async processing + manual commit

Scenario 3: Message Ordering Violated

Symptoms:

- Processed events out of sequence for same key
- Downstream system sees inconsistent state

Root Causes:

1. `max.in.flight.requests.per.connection > 1` without idempotence
2. Multiple partitions for same key (impossible)
3. Consumer multi-threaded without key-based partitioning
4. Retries on producer without idempotence

Solutions:

- ✓ `enable.idempotence=true` (implies `max.in.flight=1/5`)
- ✓ Ensure consistent key hashing across partitions
- ✓ Use partition-level processing in consumer (single thread)
- ✓ Enable idempotent producer to prevent duplicate reorders

PART 11: KAFKA INTERVIEW POWER PHRASES

Key Differentiators (For Senior/Staff Level)

Topic	Power Phrase
Kafka vs Queue	"Kafka isn't a traditional queue — it's a distributed commit log with consumer groups that maintain independent offsets, enabling replayability."
Partitioning	"Partitions are the unit of parallelism and ordering. Messages with the same key go to the same partition, preserving order per entity."
Exactly-Once	"True exactly-once requires both idempotent producers and transactional consumers, but at-least-once with idempotent processing is often the pragmatic choice."
Consumer Groups	"A consumer group provides load balancing AND fault tolerance — but maximum parallelism equals partition count."
Rebalancing	"Cooperative rebalancing is the modern approach — incremental reassignment rather than stop-the-world."

Lag Management	"Lag is the most critical metric. Monitor records-lag-max per consumer group. Zero lag doesn't always mean healthy — could mean not processing."
Compression	"lz4 or zstd for speed, gzip for space. Never use JSON without compression — Avro + zstd is the modern sweet spot."
Retries	"Retry without idempotence is dangerous. Always enable idempotent producer when retries are needed."
Kafka vs Flink	"Kafka Streams is for stateful processing within Kafka's ecosystem. Flink is for complex event processing with sub-second latency across multiple systems."

QUICK REFERENCE CARD — KAFKA

Configurations Cheat Sheet

```
# Producer – Production Settings
producer:
  acks: all
  enable.idempotence: true
  max.in.flight.requests.per.connection: 5 # if idempotence=true
  compression.type: lz4|zstd
  retries: 2147483647 # effectively unlimited
  linger.ms: 10
  batch.size: 16384

# Consumer – Production Settings
consumer:
  group.id: ${service}.${environment}
  enable.auto.commit: false
  auto.offset.reset: earliest # or latest per use case
  fetch.min.bytes: 50000
  max.poll.records: 500
  session.timeout.ms: 45000
  max.poll.interval.ms: 300000 # adjust for processing time

# Broker – Cluster Settings
broker:
  min.insync.replicas: 2
  default.replication.factor: 3
  unclean.leader.election.enable: false
  log.retention.hours: 168
  log.segment.bytes: 1073741824 # 1GB
```

Topic Design Decisions

Decision	Consideration
Partitions	Throughput per partition (~10 MB/s), rebalance time, file handles
Replication factor	3 for production, 2 for dev, 1 only for test
**min.insync.replic	